PLAN DE DESARROLLO E IMPLEMENTACIÓN DEL SOFTWARE

MaidoTouch

Sistema de Autopedido NFC con Asistente Gastronómico Inteligente

Organización	Restaurante Maido
Ubicación	Miraflores, Lima
Fecha de emisión	05 de octubre de 2026
Encargado	Ramírez Noriega, Emilio Miguel

1. Introducción

Este documento detalla las actividades planificadas para el desarrollo y la implementación de MaidoTouch, la web-app de autopedido activada por NFC y su Asistente Gastronómico Inteligente para la zona a la carta y de barra del restaurante Maido (Miraflores, Lima). Incluye las fases, tareas, cronograma, responsables, estrategia de despliegue y las consideraciones técnicas y operativas para su ejecución.

El plan parte de los documentos ya aprobados del proyecto: el Acta de Constitución (presupuesto de S/ 20,000 y plazo de 4 meses), la Matriz RACI, la Especificación de Requisitos del Software (14 requisitos funcionales y 9 no funcionales), el Diseño Arquitectónico (cliente-servidor en cuatro capas, desplegado como monolito modular) y el esquema inicial de base de datos en SQLite. El cronograma utilizado es el del panel de gestión del proyecto, con fecha de cierre el 25 de noviembre de 2026.

Dos principios del proyecto condicionan todo el plan y no admiten excepción: las alergias y restricciones alimentarias se validan siempre con reglas determinísticas y nunca con IA generativa, y el sistema complementa al mozo, nunca lo reemplaza. Por eso la implementación se diseña de forma que el restaurante pueda volver al flujo tradicional en cualquier momento sin afectar el servicio.

2. Objetivo del Plan

Definir cómo se ejecutará técnicamente el desarrollo de MaidoTouch y cómo se realizará su implementación en los entornos de pruebas (staging) y de producción, de modo que el sistema llegue al piloto del 10 de noviembre de 2026 con cero defectos críticos y con el 100 % de los casos críticos de alergias aprobados.

Objetivos específicos:

- Ordenar la construcción para que el Motor de Restricciones Alimentarias se desarrolle y pruebe antes de integrar al proveedor de IA, tal como recomienda el Diseño Arquitectónico.
- Asignar a cada tarea de implementación un responsable, un entorno y una fecha consistentes con la Matriz RACI y el cronograma del proyecto.
- Establecer reglas de control de versiones que permitan saber en todo momento qué versión del software y del esquema de base de datos está en cada entorno.
- Definir una estrategia de despliegue progresiva que proteja la experiencia premium de Maido y permita volver atrás sin interrumpir el servicio.
- Fijar el procedimiento para introducir cambios después de la puesta en producción, con validación, autorización y comunicación al equipo y al personal del restaurante.

3. Enfoque de Desarrollo

3.1 Modelo de desarrollo adoptado

Se adopta un **modelo híbrido: planificación predictiva por hitos y construcción incremental-iterativa**, apoyada en prácticas DevOps de integración continua. La planificación por hitos se mantiene porque el presupuesto, el plazo y los entregables ya están aprobados por el sponsor en el Acta de Constitución. La construcción se hace por incrementos funcionales de una a dos semanas, cada uno con una versión desplegable en staging y una revisión al cierre.

Se descartó la cascada pura porque la integración con el proveedor externo de IA y la instalación del NFC tienen incertidumbre técnica (riesgos RP03 y RP09) que conviene descubrir temprano. Se descartó un Scrum completo porque el equipo de desarrollo es reducido —un Programador-Tester apoyado por el Responsable de Datos e IA— y las ceremonias completas consumirían tiempo que el plazo de 4 meses no permite.

3.2 Incrementos planificados

Incremento	Periodo	Contenido principal	Versión
Incremento 1 · Núcleo	28/09 – 15/10/2026	Web-app base (activación por token, catálogo bilingüe, perfil de preferencias, carrito), Motor de Restricciones Alimentarias y back-office de disponibilidad.	v0.1.0
Incremento 2 · Asistente	28/09 – 25/10/2026	Adaptador del proveedor de IA con timeout y fallback por reglas, Motor de Recomendación, generador de explicaciones y trazabilidad de recomendaciones.	v0.2.0
Incremento 3 · Operación	16/10 – 30/10/2026	Tags NFC y QR por puesto, envío de comandas a cocina/barra, llamado al mozo y derivación al personal.	v0.3.0
Estabilización	31/10 – 09/11/2026	QA integral, corrección de defectos, medición de latencia y versión candidata.	v1.0.0-rc

3.3 Herramientas de desarrollo

Ámbito	Herramienta	Uso en el proyecto
Control de versiones	Git + repositorio privado en GitHub	Código fuente, scripts de base de datos, documentación técnica y etiquetas de versión.
Gestión del trabajo	GitHub Issues y Projects (tablero)	Backlog por requisito (RF/RNF), seguimiento de tareas, defectos y solicitudes de cambio.
Integración continua	GitHub Actions	Ejecución automática de pruebas en cada pull request y despliegue a staging al integrar en develop.
Editor	Visual Studio Code	Desarrollo de la web-app y de la API REST.
Base de datos	SQLite (esquema.sql)	Persistencia del catálogo, alérgenos, sesiones, pedidos, comandas y métricas del piloto, con PRAGMA foreign_keys activado en cada conexión.
Pruebas de API	Postman	Colecciones de pruebas de los servicios de sesión, catálogo, recomendación y pedido.
Prototipo UX	Figma	Prototipo de experiencia de usuario validado por el cliente antes de construir las pantallas.
Infraestructura	Servidor único en la nube con HTTPS/TLS	Entornos de staging y producción dentro de la partida de S/ 1,500 de OPEX.

3.4 Flujo de trabajo propuesto

1. Cada requisito del SRS se registra como una tarea en el tablero, con su criterio de aceptación tomado del Acta de Constitución.
2. El Programador-Tester o el Responsable de Datos e IA crea una rama feature/ desde develop y desarrolla la tarea con sus pruebas unitarias.
3. Abre un pull request; la integración continua ejecuta las pruebas y otro integrante revisa el código. Todo cambio en el Motor de Restricciones requiere además la revisión de la Responsable de Calidad.
4. Al aprobarse, la rama se integra en develop y se despliega automáticamente en staging.
5. Al cierre de cada incremento se hace una demostración en staging con el Director de Proyecto y, cuando corresponde, con el Cliente; lo aprobado se etiqueta como versión.

4. Fases del Desarrollo

Las fases siguen el cronograma del proyecto. La fase 2 es precondición dura de las fases 3 y 4: sin catálogo con ingredientes y alérgenos validados (paquete de trabajo 1.2.2 de la EDT), el Motor de Restricciones no puede probarse.

Fase del Desarrollo	Actividades Clave	Entregables
1. Análisis y requisitos 08/09 – 14/09/2026	• Levantamiento de requisitos con el cliente. • Definición de alcance y EDT. • Especificación de 14 RF y 9 RNF.	• SRS v1.0 • Acta de Alcance • EDT y Diccionario
2. Diseño y preparación de datos 15/09 – 27/09/2026	• Diseño arquitectónico y diagramas UML. • Diseño del esquema de base de datos SQLite. • Levantamiento de la línea base (tiempos, ticket, consultas al mozo). • Validación del catálogo, ingredientes y alérgenos con cocina. • Prototipo de experiencia de usuario.	• Diseño Arquitectónico v1.0 • esquema.sql (27 tablas en SQLite) • Catálogo piloto 100 % validado • Prototipo validado por el cliente • Informe de línea base
3. Codificación del núcleo 28/09 – 15/10/2026	• Web-app responsive ES/EN: activación, catálogo, perfil, carrito. • API REST sobre HTTPS. • Motor de Restricciones con reglas determinísticas y estado Seguro / Inseguro / Sin dato. • Back-office de disponibilidad con historial.	• Web-app base v0.1.0 en staging • Suite de pruebas del motor con los casos críticos de alergias aprobados
4. Integración del asistente de IA 28/09 – 25/10/2026	• Adaptador del proveedor de IA (timeout de 5 s, validación contra el catálogo enviado). • Estrategia de fallback por reglas. • Generador de explicaciones. • Registro de recomendaciones y eventos de aceptación/rechazo.	• Asistente v0.2.0 en staging • Informe de KPIs técnicos (interpretación ≥ 90 %, 0 % información inventada)
5. Integración con la operación 16/10 – 30/10/2026	• Programación de tags NFC y códigos QR por mesa y puesto de barra. • Envío de comandas a la pantalla de cocina/barra. • Llamado al mozo y derivación por restricción no validable.	• Versión v0.3.0 en staging • NFC/QR operativo en ≥ 99 % de puestos • Flujo aprobado por cocina/barra
6. Pruebas (QA integral) 31/10 – 09/11/2026	• Pruebas funcionales de RF01–RF14. • Pruebas de seguridad alimentaria y de información inventada. • Medición de latencia P95 en red 4G. • Pruebas de regresión y de fallback sin IA.	• Informe de QA con evidencias • 0 defectos críticos abiertos • Versión candidata v1.0.0-rc
7. Implementación y piloto 31/10 – 23/11/2026	• Capacitación del personal. • Paso a producción y despliegue progresivo. • Piloto de 2 semanas con monitoreo y medición.	• v1.0.0 en producción • Registro de capacitación • Informe del piloto
8. Cierre 24/11 – 25/11/2026	• Entrega de documentación técnica y manuales. • Aceptación formal del cliente y del sponsor.	• Acta de aceptación firmada • Manual de operación

Descripción de columnas: Fase del Desarrollo: etapa principal y su periodo. Actividades Clave: tareas realizadas en esa etapa. Entregables: resultados esperados.

5. Plan de Implementación

Las tareas se ordenan cronológicamente. Los responsables son los mismos que figuran como «R» en la Matriz RACI; el Director de Proyecto aprueba el paso a producción.

Tarea de Implementación	Responsable	Entorno Objetivo	Fecha Estimada
Configurar el repositorio, la estrategia de ramas y el pipeline de integración continua	Ramírez Noriega, Emilio Miguel	Desarrollo	02/10/2026
Provisionar el servidor en la nube, el dominio y el certificado HTTPS para staging y producción	Ramírez Noriega, Emilio Miguel	Staging y Producción	09/10/2026
Crear la base de datos ejecutando esquema.sql sobre una base nueva y verificar las claves foráneas	Paredes Aguirre, Jean Franco	Staging	09/10/2026
Cargar el catálogo validado: productos, textos ES/EN, ingredientes, alérgenos y evaluación de restricciones	Paredes Aguirre, Jean Franco	Staging	15/10/2026
Desplegar la web-app base y el Motor de Restricciones (v0.1.0)	Ramírez Noriega, Emilio Miguel	Staging	15/10/2026
Configurar las credenciales del proveedor de IA como variables de entorno y desplegar el asistente con fallback (v0.2.0)	Paredes Aguirre, Jean Franco	Staging	25/10/2026
Registrar mesas y puestos de barra, generar tokens y programar los tags NFC con su QR de respaldo	Ramírez Noriega, Emilio Miguel	Producción (local)	30/10/2026
Instalar la pantalla de comandas en cocina y barra y probar el flujo de pedido	Ramírez Noriega, Emilio Miguel	Producción (local)	30/10/2026
Ejecutar el QA integral con evidencias sobre la versión candidata	Rodríguez Cardozo, Evelyn Victoria	Staging	09/11/2026
Capacitar al personal de sala, cocina, barra y al encargado de carta	Arroyo Mera, Anghelo	Producción (local)	09/11/2026
Crear las cuentas del personal en el back-office con sus roles	Ramírez Noriega, Emilio Miguel	Producción	09/11/2026
Reunión Go/No-Go y paso a producción de la versión v1.0.0	Valera Lizárraga, Francisco Kadin (aprueba) · Ramírez Noriega (ejecuta)	Producción	09/11/2026
Ejecutar el piloto de 2 semanas con monitoreo diario de incidencias y latencia	Paredes Aguirre, Jean Franco	Producción	10/11 – 23/11/2026
Consolidar las métricas del piloto (aceptación, CSAT, tiempos) y emitir el informe	Paredes Aguirre, Jean Franco	Producción	23/11/2026
Entregar documentación y obtener la aceptación formal	Valera Lizárraga, Francisco Kadin	Producción	25/11/2026

Descripción de columnas: Tarea de Implementación: actividad para desplegar el software. Responsable: encargado de ejecutarla. Entorno Objetivo: desarrollo, staging o producción. Fecha Estimada: fecha límite de la tarea.

Criterios de Go/No-Go para el paso a producción. La versión v1.0.0 solo se libera si se cumplen todos: 0 defectos críticos abiertos; 100 % de los casos críticos de alergias aprobados; catálogo piloto 100 % validado; NFC/QR operativo en al menos el 99 % de los puestos; latencia del asistente ≤ 3 s en P95; fallback sin IA probado; y 100 % del personal clave capacitado. Si falta uno, el piloto se posterga y se informa al sponsor.

6. Plan de Control de Versiones

El software se gestiona con **Git** en un repositorio privado de **GitHub**. Nada llega a producción sin pasar por una rama, un pull request revisado y una etiqueta de versión.

6.1 Estrategia de ramas

Rama	Propósito	Reglas
main	Código en producción.	Protegida: solo recibe merges desde release/ o hotfix/ con aprobación del Director de Proyecto. Cada merge lleva etiqueta de versión.
develop	Integración del trabajo en curso; se despliega en staging.	Recibe pull requests desde feature/ y fix/ con pruebas en verde y una revisión.
feature/RFxx-descripcion	Desarrollo de una funcionalidad. Ej.: feature/RF14-derivacion-personal.	Nace de develop y se elimina al integrarse. Vida corta (máximo una semana).
fix/	Corrección de defectos detectados en staging.	Mismas reglas que feature/.
release/x.y.z	Estabilización de una versión candidata durante el QA.	Solo admite correcciones; al aprobarse se integra en main y en develop.
hotfix/x.y.z	Corrección urgente en producción.	Nace de main; se integra en main y en develop. Ver sección 8.

6.2 Numeración de versiones

Se usa versionado semántico **MAYOR.MENOR.PARCHE**. La versión MAYOR cambia ante cambios incompatibles (por ejemplo, un nuevo esquema de base de datos que exige migrar datos); la MENOR, al agregar funcionalidad compatible; y el PARCHE, al corregir defectos. Durante el desarrollo se trabaja en la serie 0.x y la primera versión en producción es la 1.0.0.

Versión	Hito	Fecha estimada
v0.1.0	Web-app base y Motor de Restricciones	15/10/2026
v0.2.0	Asistente de IA con fallback y trazabilidad	25/10/2026
v0.3.0	NFC/QR, comandas a cocina/barra y llamado al mozo	30/10/2026
v1.0.0-rc.N	Versiones candidatas durante el QA integral	31/10 – 09/11/2026
v1.0.0	Paso a producción para el piloto	09/11/2026
v1.0.x	Correcciones durante el piloto	10/11 – 23/11/2026

6.3 Etiquetado, commits y base de datos

- **Etiquetas:** cada versión liberada se marca con una etiqueta anotada (git tag -a v1.0.0) y se documenta en un archivo CHANGELOG.md con los requisitos y defectos que incluye.
- **Mensajes de commit:** formato Conventional Commits con referencia al requisito o al issue (por ejemplo, «feat(RF09): ranking por reglas en fallback»).
- **Esquema de base de datos:** esquema.sql se versiona en el repositorio como migración inicial (001). Todo cambio posterior se escribe como un script de migración numerado y nunca se modifica una migración ya aplicada. Antes de aplicar una migración en producción se respalda el archivo SQLite.
- **Datos de alérgenos:** los cambios en ingredientes, alérgenos y evaluación de restricciones no se hacen desde el código, sino por el procedimiento de validación, que deja registro de quién validó y cuándo.
- **Secretos:** las claves del proveedor de IA y las credenciales del servidor no se suben al repositorio; se gestionan como variables de entorno.

7. Estrategia de Despliegue

Se utilizará un **despliegue progresivo, operado en paralelo con el flujo tradicional del mozo**. No se usa big bang porque un error de seguridad alimentaria o un fallo en hora punta afectaría a todos los comensales a la vez y dañaría la percepción del servicio premium (riesgo RPR12). Tampoco hay un sistema anterior que reemplazar: el «sistema paralelo» es el pedido tradicional con el mozo, que sigue disponible durante todo el piloto y después de él.

7.1 Etapas del despliegue

Etapa	Periodo	Alcance	Criterio para avanzar
0 · Ensayo general	09/11/2026	Personal de Maido simula pedidos reales en el local, sin comensales.	Flujo completo sin errores y comandas recibidas al 100 %.
1 · Barra	10/11 – 12/11/2026	Solo los puestos de barra, en el servicio de menor demanda, con un integrante del equipo presente.	0 incidentes de seguridad alimentaria y 0 comandas perdidas.
2 · Barra y mesas	13/11 – 16/11/2026	Se agregan las mesas de la zona a la carta en almuerzo y cena.	Latencia ≤ 3 s en P95 y disponibilidad estable durante el servicio.
3 · Operación completa	17/11 – 23/11/2026	Toda la zona a la carta y de barra; medición de métricas del piloto.	Muestra mínima de comensales completada.

7.2 Cómo se garantiza una transición segura

- **Activación por puesto:** cada mesa o puesto de barra tiene su propio token de acceso que puede activarse o desactivarse. Así se habilitan los puestos por etapas y, ante un problema en uno, se apaga solo ese puesto y vuelve al pedido tradicional.
- **Mozo siempre disponible:** el botón de llamado al mozo está en todas las pantallas y el personal de sala está capacitado para tomar el pedido manualmente si la app falla (RNF03).
- **Degradación sin IA:** si el proveedor de IA no responde en 5 s, el sistema recomienda con reglas sobre la misma lista segura; el servicio no se detiene (RPR05).
- **Reversión rápida:** antes de cada despliegue se respalda la base de datos y se conserva la versión anterior etiquetada. Si la nueva versión falla, se reinstala la anterior y se restaura el respaldo en menos de 30 minutos.
- **Despliegues fuera del servicio:** las actualizaciones se aplican antes de la apertura o después del cierre, nunca durante el almuerzo o la cena.
- **Monitoreo diario:** durante el piloto se revisan cada día los errores, la latencia del asistente, las comandas enviadas y las derivaciones al personal, y se comunica un resumen al Director de Proyecto.

8. Gestión de Cambios en Producción

Todo cambio sobre la versión en producción sigue un procedimiento formal a cargo del Director de Proyecto (riesgo RP02 del Acta de Constitución). Activar o desactivar la disponibilidad de un plato desde el back-office no es un cambio de software: es una operación diaria del encargado de carta.

8.1 Tipos de cambio

Tipo	Ejemplos	Quién autoriza	Ventana
Menor	Corrección de un texto, una foto o un estilo; defecto sin impacto funcional.	Director de Proyecto	Siguiente ventana fuera de servicio.
Normal	Nueva funcionalidad o ajuste de una regla de negocio o del ranking.	Director de Proyecto; el Sponsor si afecta alcance, costo o plazo.	Ventana programada, con aviso de 24 h.

Tipo	Ejemplos	Quién autoriza	Ventana
Datos críticos	Cambio de receta, ingrediente o alérgeno de un plato.	Responsable de Datos con validación de cocina; revisión de Calidad.	Antes del siguiente servicio.
Emergencia	Fallo que impide pedir o riesgo de recomendación insegura.	Director de Proyecto (puede ser verbal y se documenta en 24 h).	Inmediata.

8.2 Procedimiento

1. **Solicitud.** Quien detecta la necesidad la registra como issue «Solicitud de cambio» en GitHub, con descripción, motivo y urgencia.
2. **Análisis de impacto.** El Analista-Diseñador y el Programador-Tester estiman el esfuerzo y el impacto en requisitos, base de datos, costo y cronograma.
3. **Autorización.** Según el tipo de cambio, lo aprueba el Director de Proyecto o el Sponsor. Si toca el Motor de Restricciones o los datos de alérgenos, también firma la Responsable de Calidad.
4. **Desarrollo.** Se implementa en una rama propia con sus pruebas, siguiendo el plan de control de versiones.
5. **Validación.** Se prueba en staging: el caso del cambio, la regresión completa y, siempre, los casos críticos de alergias.
6. **Despliegue.** Se respalda la base de datos y se despliega en la ventana acordada con una nueva etiqueta de versión.
7. **Comunicación.** Se informa al equipo y al personal de Maido qué cambió y desde cuándo, mediante una nota de versión breve y el grupo de coordinación del restaurante.
8. **Verificación y cierre.** Se verifica el cambio en el siguiente servicio y se cierra la solicitud con evidencia.

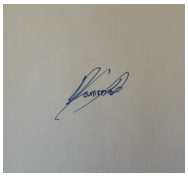
8.3 Regla especial para la seguridad alimentaria

Si se detecta que un plato tiene un ingrediente o alérgeno mal registrado, primero se desactiva el plato desde el back-office y luego se corrige el dato. Un plato cuya receta cambia pierde su validación y queda como «sin dato» hasta que se valide de nuevo; mientras tanto, el sistema no lo recomienda a comensales con restricciones y deriva la consulta al personal. Ningún cambio puede desactivar o saltarse el Motor de Restricciones.

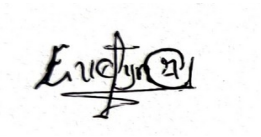
9. Aprobaciones y Validaciones del Documento

Esta sección incluye las firmas de las personas que aprueban y validan el documento.

Programador y Tester (responsable técnico del desarrollo)
Nombre: Ramírez Noriega, Emilio Miguel

Firma:  Fecha:07/10/2026

Responsable de Calidad
Nombre: Rodríguez Cardozo, Evelyn Victoria

Firma:  Fecha:07/10/2026

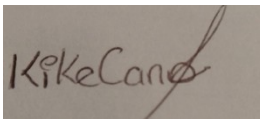
Director de Proyecto

Nombre: Valera Lizárraga, Francisco Kadin

Firma:  Fecha:07/10/2026

Cliente

Nombre: Cano Asto, Jorge Enrique

Firma:  Fecha: 07/10/2026

10. Conclusiones

El plan organiza el desarrollo de MaidoTouch en un modelo híbrido: hitos aprobados por el sponsor y construcción por incrementos, cada uno desplegado y revisado en staging. Esto permite descubrir pronto los riesgos técnicos del NFC y del proveedor de IA sin perder el control del plazo de 4 meses ni del presupuesto de S/ 20,000.

El orden de construcción respeta la decisión central del diseño: primero el catálogo con alérgenos validados, luego el Motor de Restricciones con sus casos críticos aprobados y solo después la integración con la IA, con el fallback desarrollado en el mismo incremento. Así, la seguridad alimentaria queda probada antes de que exista cualquier recomendación.

La implementación progresiva, operada en paralelo con el mozo, junto con la activación por puesto y la reversión en menos de 30 minutos, asegura que ninguna falla deje al comensal sin atención. El control de versiones con Git y el procedimiento de gestión de cambios dan trazabilidad a cada versión y a cada cambio, y garantizan que ninguna modificación pueda saltarse las reglas de alergias.

11. Aprendizaje del Equipo

Aspecto Aprendido	Comentario del Equipo
El orden de construcción también es una decisión de seguridad	Al planificar entendimos que no basta con que el Motor de Restricciones exista: tiene que construirse y probarse antes que la IA. Si lo dejábamos para después, el asistente podría haberse probado recomendando platos sin filtro, justo lo que el proyecto prohíbe.
Implementar no es lo mismo que desplegar	Al principio pensamos la implementación como «subir el sistema al servidor». El plan nos mostró que también incluye instalar los tags, la pantalla de cocina, cargar datos validados, crear cuentas y capacitar al personal. Sin eso, el software funciona pero el restaurante no puede usarlo.
El flujo tradicional es nuestro plan de respaldo	Como no hay un sistema anterior que reemplazar, descubrimos que el mozo cumple el papel del sistema en paralelo. Diseñar la activación por puesto nos permite apagar una mesa con problemas sin afectar a las demás.
Versionar también la base de datos	Nos dimos cuenta de que el esquema SQL y los datos de alérgenos cambian igual que el código. Si no se versionan con migraciones y respaldos, no hay forma de saber qué versión del catálogo estaba activa cuando ocurrió un incidente.

Aspecto Aprendido	Comentario del Equipo
Un cambio pequeño puede ser crítico	Cambiar un ingrediente de un plato parece una tarea menor, pero en MaidoTouch afecta la seguridad de un comensal alérgico. Por eso definimos los cambios de datos críticos como un tipo propio, con validación de cocina y revisión de Calidad.
Criterios claros para decidir el paso a producción	Fijar los criterios de Go/No-Go con números concretos evita que la decisión dependa del entusiasmo del equipo o de la presión del cronograma. Si un criterio no se cumple, el piloto se posterga y se informa al sponsor.

Descripción de columnas: Aspecto Aprendido: tema o habilidad desarrollada al elaborar el plan. Comentario del Equipo: reflexión sobre su importancia o utilidad.